



# AdFlush: A Real-World Deployable Machine Learning Solution for Effective Advertisement and Web Tracker Prevention

Kiho Lee\*  
Sungkyunkwan University  
Republic of Korea  
kiho@skku.edu

Chaejin Lim\*  
Sungkyunkwan University  
Republic of Korea  
chaejin98@skku.com

Beomjin Jin  
Sungkyunkwan University  
Republic of Korea  
jinbumjin@skku.edu

Taeyoung Kim  
Sungkyunkwan University  
Republic of Korea  
tykim0402@skku.edu

Hyoungshick Kim  
Sungkyunkwan University  
Republic of Korea  
hyoung@skku.edu

## ABSTRACT

Conventional ad blocking and tracking prevention tools often fall short in addressing web content manipulation. Machine learning approaches have been proposed to enhance detection accuracy, yet aspects of practical deployment have frequently been overlooked. This paper introduces AdFlush, a novel machine learning model for real-world browsers. To develop AdFlush, we evaluated the effectiveness of 883 features, ultimately selecting 27 key features for optimal performance. We tested AdFlush on a dataset of 10,000 real-world websites, achieving an F1 score of 0.98, thereby outperforming AdGraph (F1 score: 0.93), WebGraph (F1 score: 0.90), and WTAGraph (F1 score: 0.84). Additionally, AdFlush significantly reduces computational overhead, requiring 56% less CPU and 80% less memory than AdGraph. We also assessed AdFlush's robustness against adversarial manipulations, demonstrating superior resilience with F1 scores ranging from 0.89 to 0.98, surpassing the performance of AdGraph and WebGraph, which recorded F1 scores between 0.81 and 0.87. A six-month longitudinal study confirmed that AdFlush maintains a high F1 score above 0.97 without the need for retraining, underscoring its effectiveness.

## CCS CONCEPTS

• Security and privacy → Web application security.

## KEYWORDS

Ad blocking; Web tracking; Machine learning; Web security

## ACM Reference Format:

Kiho Lee\*, Chaejin Lim\*, Beomjin Jin, Taeyoung Kim, and Hyoungshick Kim. 2024. AdFlush: A Real-World Deployable Machine Learning Solution for Effective Advertisement and Web Tracker Prevention. In *Proceedings of the ACM Web Conference 2024 (WWW '24)*, May 13–17, 2024, Singapore, Singapore. ACM, Singapore, 12 pages. <https://doi.org/3589334.3645698>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).  
WWW '24, May 13–17, 2024, Singapore, Singapore.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 979-8-4007-0171-9/24/05.  
<https://doi.org/3589334.3645698>

## 1 INTRODUCTION

Many internet users are concerned about web advertisements (ads) and trackers that display unsolicited content or collect their personal information without explicit consent. Several tools are available to address these issues, including Adblock Plus, uBlock Origin, and Ghostery. These tools utilize filter lists such as EasyList [1] and EasyPrivacy [2], which contain rules for determining whether HTTP requests are related to ads or trackers. However, manual maintenance of these filter lists requires significant human effort, and they are prone to false-positive and false-negative errors [3]. The rules primarily rely on specific patterns, which renders them ineffective against evasive or manipulated content.

Machine learning (ML) detection techniques [4–7] have recently been investigated to address the limitations of traditional filter list-based methodologies. AdGraph [5] and WTAGraph [7] construct graphs to extract significant features based on HTTP traffic. Furthermore, WebGraph [6] utilizes additional behavioral features to build more robust detection models against adversarial evasion techniques. However, these studies have mainly focused on developing highly accurate detection models by incorporating new features to identify ads and trackers. While ML models show great promise in identifying them, they often rely on extensive feature sets. This can lead to significant execution time and memory overhead for feature extraction, model training, and inference. This may not be suitable for browser settings on a typical desktop machine due to limited computational resources.

To deploy ML detection models in standard web browsers, we need to identify a minimal yet effective set of features that can be efficiently extracted without compromising accuracy. This involves studying the features used by existing models, analyzing their relevance, and refining the feature set based on considerations such as computational complexity and feasibility in a browser environment.

To achieve this goal, we carefully examined all potential features from prior models and analyzed their impact on detection accuracy and feasibility in web browser settings. Based on our feature engineering, we selected an optimal set of features for detecting advertisements and web trackers. We then used automated machine learning (AutoML) [8] on these features to identify the most efficient machine model, AdFlush.

To demonstrate the effectiveness of AdFlush, we evaluate the performance of AdFlush against state-of-the-art models (AdGraph

\*Both authors contributed equally to this research.

[5], WebGraph [6], and WTAGraph [7]) using the following performance evaluation metrics: F1 score, training time, inference time, memory usage, and CPU usage. AdFlush achieves an F1 score of 0.98, outperforming AdGraph (F1 score: 0.93), WebGraph (F1 score: 0.90), and WTAGraph (F1 score: 0.84). The average inference time for AdFlush over 830K traffic samples is 2.3 seconds, considerably faster than AdGraph's 17.4 seconds. AdFlush has demonstrated excellent computational efficiency by exhibiting 56% lower CPU usage and 80% less memory consumption than AdGraph. Regarding its robustness against adversarial attacks such as URL manipulation and JavaScript obfuscation, AdFlush significantly outperforms AdGraph and WebGraph's performance.

Unlike previous research, we consider the feasibility of the features within real-world browser settings, demonstrating that AdFlush can be deployed as a web browser extension without causing significant CPU or memory overheads. Furthermore, AdFlush does not transmit User-Agent values or cookies to third parties, upholding a strong commitment to user privacy. Our main contributions can be summarized as follows:

- We propose a new set of 27 features for detecting ads and trackers in real-world browser extension environments. The feature set was carefully selected from a pool of 883 features, including 350 existing features from state-of-the-art methods [5–7] and 533 new JavaScript features that are robust to adversarial samples with JavaScript obfuscation.
- We developed AdFlush, a lightweight yet highly accurate model for detecting advertisements and web trackers using AutoML [8]. To demonstrate its practicality, we implemented AdFlush as a Chrome extension. The source code and dataset for AdFlush are available at <https://github.com/SKKU-SecLab/AdFlush>. We have also made a comprehensive video demo available at <https://youtu.be/dzdfqpiCjKg>, demonstrating AdFlush's practicality in real-world web browser settings.
- We evaluated the long-term effectiveness and adversarial robustness of AdFlush. In a six-month longitudinal study, AdFlush maintained a high F1 score above 0.97 without retraining, demonstrating its effectiveness. Commercial filter lists lagged by an average of 80 days in updating ads and trackers, while AdFlush detected 642 URLs across 108 domains that were missed by filter lists. Benchmarked against state-of-the-art models using three adversarial test cases (URL manipulation, JavaScript obfuscation, and ML-based adversarial sample generation), AdFlush consistently outperformed other models.

## 2 RELATED WORK

### 2.1 Risks of Advertisements and Trackers

Web advertising and tracking facilitate personalized experiences but pose significant privacy and security challenges [9]. Trackers create extensive user profiles by monitoring activities such as site visits and clicks for targeted advertising, often distributing this information without consent, which increases the risks of data breaches and identity theft [10, 11]. Furthermore, *malvertising* presents a risk by distributing malware through advertisements, affecting even sites considered safe and thus amplifying users' vulnerability [12, 13]. Concerns are also raised by tracking technologies like cookies and browser fingerprinting [14, 15].

### 2.2 Filter List-Based Approaches

Many web users rely on browser extensions such as Adblock Plus, uBlock Origin [16], Privacy Badger [17], and Disconnect [18] to protect themselves from intrusive ads and privacy-invading web trackers. Recognizing the importance of these ad blockers, several web browsers, including Firefox [19], Ghostery, and Brave [20], now offer built-in ad-blocking solutions. This demonstrates the importance of these extensions in the browsing environment.

Ad blocking solutions rely on filter lists like EasyList [1], EasyPrivacy [2], and Fanboy's List [21], which are manually curated to block websites and scripts. Despite their role in blocking harmful content, these lists have drawbacks [22]. They are labor-intensive and difficult to scale, especially across numerous websites. Moreover, the crowdsourcing approach can lead to regional bias, with less represented areas having fewer ad-blocking rules [23]. Additionally, filter lists often lag behind the evolving tactics of web trackers, like script obfuscation [24].

### 2.3 ML-Based Approaches

To overcome the limitations of traditional methods that rely on manually created filter lists, various techniques have been proposed for generating filter lists in an automated manner. For instance, Bhagavatula *et al.*'s model [25] streamlines the process of creating and maintaining filter lists. Recently, AutoFR [26] introduced an efficient method for generating filter lists using reinforcement learning. However, it focuses solely on image ads and does not prevent trackers that are not displayed on web pages.

Recent graph-based ML approaches, such as AdGraph [5], PageGraph [23], WTAGraph [7], and WebGraph [6], utilize features from ads and trackers' interactions across HTML, HTTP, and JavaScript layers to train models for detection. PageGraph extends AdGraph's graph representation by refining event attribution and capturing more behaviors, but both overlook HTTP redirects and request header information. WebGraph [6] addresses this by adding features that trace information flow within a browser, including HTTP redirects and request headers. Given the natural graph structure of HTTP network traffic, WTAGraph leverages a graph neural network (GNN) model for better detection accuracy. However, both WebGraph and WTAGraph operate offline only in practice due to performance overheads, which subjects them to challenges similar to traditional filter lists. Notably, WebGraph's use of over 3,000 features without GPU acceleration results in lengthy inference times, making real-time desktop operation impractical for average users.

Another approach involves utilizing computer vision techniques to detect ad images. Storey *et al.* [27] introduced Ad Highlighter, the inaugural perceptual ad-blocker that identifies ad disclosures by merging web-filtering rules with computer vision methods. Due to the enhanced robustness of perceptual techniques, many contemporary ad-blockers have integrated similar concepts. For instance, Adblock Plus has adopted image-matching filters. Percival [28] employs CNN classification models for advertising image detection. However, challenges arise when ads are placed outside of iframes, use transparent logos, or are fragmented into smaller images. These tactics hinder the efficacy of computer vision in detecting ads [29].

ML-based approaches have potential for ad-blocking tools, but they have not been widely adopted yet due to performance and

reliability issues. Performance-wise, the overheads associated with feature instrumentation and executing ML pipelines in real-time can offset the benefits of ad blocking [26]. Reliability issues arise from doubts about the robustness of ML models in real-world settings [28]. Additionally, recent graph-based ML models typically aggregate features only after webpages have fully loaded, potentially allowing ads and trackers to execute before feature extraction. Evaluations often neglect practical considerations, including the model's impact on webpage load time, CPU and memory overhead, and robustness against adversarial attacks.

Therefore, we propose a practical ML-based solution designed for seamless integration as a browser extension to address these limitations. We aim to achieve high detection accuracy using only practical features supported by commercial web browsers.

### 3 FEATURE ANALYSIS

This section comprehensively analyzes 883 potential features, including 350 used by existing ML-based methods and 533 new features that are robust to adversarial samples with JavaScript obfuscation. We assess the feasibility and contribution of each feature to overall detection efficacy, identifying key features that can be practically implemented in web browsers.

#### 3.1 Data Collection

To collect a comprehensive dataset of real-world websites, we sampled the top 10,000 websites ranked by Tranco [30]. This was because Alexa, previously used in studies for AdGraph [5] and WebGraph [6], no longer provides an accessible or updated ranking service. We also followed the approach of WTAGraph [7], which used the top 10,000 sites based on Tranco's ranking.

We set a 60-second timeout for each website to account for the variability in complexity and structure of different websites. Our data collection process resulted in an average crawl time of 32.58 seconds, with a standard deviation (SD) of 11.26 seconds, indicating that a 60-second timeout is appropriate for most sites. With seven web crawler instances running in parallel for 11 hours using OpenWPM [24], we generated a total of 830,160 requests and made them publicly available at <https://zenodo.org/records/10673857>. We split the dataset into an 8:2 ratio for training and testing. This ensures that the model is trained on a representative sample of the data and evaluated on a held-out set that it has not seen before.

#### 3.2 Candidate Features

We thoroughly analyzed the potential features present in our gathered dataset, integrating all features leveraged by AdGraph [5], WebGraph [6], and WTAGraph [7]. This resulted in a total of 3,323 distinct features, with some overlap.

AdGraph focuses on monitoring DOM tree alterations from JavaScript executions or HTTP requests. The resulting changes are formulated into a graph, from which 28 features are derived. These features encompass both the graph's structural attributes and content-based features, such as the attributes of the request URL.

WebGraph builds upon AdGraph by representing a broader spectrum of components in its graph structure, including HTML, network, JavaScript, and storage components. Drawing from this graph,

WebGraph uses 59 features that encapsulate both structural attributes and advanced dataflow characteristics, with special emphasis on cookie access and information sharing between nodes.

WTAGraph constructs an attribute homogeneous multi-graph (AHMG) from visited web pages, translating it into a feature vector, including character strings, JavaScript APIs, and HTTP request metrics. In AdFlush, we condensed WTAGraph's vector space of 3,200 URL character representations into 30 features for FQDNs and 200 for request URLs using continuous bag-of-words features, balancing computational feasibility and robustness against overfitting.

Furthermore, we introduced 533 features to enhance AdFlush's robustness against JavaScript obfuscation techniques. Existing features are insufficiently resilient against adversarial samples employing JavaScript obfuscation. These new features include  $n$ -gram frequencies parsed from the abstract syntax tree (AST), the structure of the AST, and various script-based metrics (detailed in Appendix A). They stem from understanding code behavior and the intricacies of obfuscation tactics, which complicate the task for ML-based detectors in identifying malicious JavaScript code [31].

We present a comprehensive feature list by combining 350 features from three existing models [5–7] with 533 additional JavaScript-specific features. Some features, such as URL character embeddings, are combined into a single feature label for clarity. We used the code provided in the research to extract features whenever possible. Otherwise, we implemented the code from scratch, especially for proprietary features or when the source code from the baseline models was unavailable.

#### 3.3 Feature Categorization

As presented in Appendix A, we categorize our features into four groups: JavaScript, URL, HTTP header, and Graph as follows:

- **JavaScript** features are derived from the HTML structure of a web page, focusing on functions within `<script>` tags and JavaScript code. They also monitor specific API usage, revealing downstream behavior of advertising and tracking actions. For example, `eval()` can trigger hidden `<script>` tags to collect user tracking data, while code tracking significant user interaction often pertains to advertising.
- **URL** features represent values from source and request URLs. Keywords in the request URL or its length can provide insights into the request's purpose. Though ad and tracker services use various URL manipulation techniques, embedding URLs can effectively counter these methods.
- **HTTP header** features are derived from values in HTTP request headers. These encompass content policy type, User-Agent strings, referrer data, and other metadata embedded in the HTTP request header. These features are crucial as they offer context about a request, often containing user-identifying information or insights about the request's nature.
- **Graph** features illustrate interactions between different graph nodes, representing various web page elements, with edges representing interactions between them. Potentially important features include the distance between nodes, node centrality, and the number of requests or redirects involving a node. These features have recently received increased research attention due to their potential to highlight the complex dynamics of various components on web pages.

In our categorization, AdGraph primarily utilizes URL and Graph features, incorporating 1, 11, 1, and 15 features from each category, respectively. WebGraph emphasizes Graph features, using 7 JavaScript and 52 Graph features. WTAGraph primarily uses 32 JavaScript, 3,200 URL, and 34 HTTP header features to construct a GNN structure with these features.\*

We evaluated the feasibility of integrating each feature into standard web browsers, according to the following rules.

- **Rule 1 (Avoiding loading additional resources):** Feature extraction should not require executing JavaScript, WebAssembly, or loading extra browser elements. As AdFlush aims to intercept ads and trackers before they load, it should depend solely on features that avoid inspecting interactions that could introduce the user to undesired ads or trackers.
- **Rule 2 (Avoiding using multiple requests):** Feature values should be derived from a single web page request to ensure swift detection and real-time analysis without excessive memory or storage use. Nested queries on various requests demand user history and significant computation.

Under these rules, most Graph features are infeasible because they violate **Rule 1** or **Rule 2**. For example, the *number\_of\_set\_then\_get\_storage* feature observes data sharing nodes in order, requires JavaScript execution with timestamps and violates **Rule 1**. The more complex *indirect\_all\_average\_degree\_connectivity* feature cannot be computed without fetching all nodes and edges, violating **Rule 2** because it requires access to the user's site visit history about all connected nodes, raising potential privacy concerns.

The only Graph feature efficiently computed with a single request is *is\_parent\_script*. This feature determines whether the initiator of the request is a JavaScript resource. Therefore, *is\_parent\_script* is the only Graph feature we consider feasible to implement in real-world browser settings. As a result, we excluded 52 Graph features out of the 350 features from our candidate feature set.

### 3.4 Key Feature Identification

To identify the most crucial ad and web tracking detection features, we evaluated the relevance, impact, and association with ad/tracker detection of 298 established state-of-the-art features and 533 newly proposed features. We identified key features from each group separately, including the 533 newly proposed features, even though they exhibited lower importance in normal samples.

We first examined the correlation between each feature and the labels using the point-biserial correlation coefficient [32]. We discarded 7 and 178 features from the existing and new sets, respectively, with a  $p$ -value exceeding 0.1, indicating that these features have an insignificant correlation with the labels and are, therefore, unlikely to be relevant to the model.

We next curated the key feature sets for AdFlush from the remaining 291 existing and 355 new features. We used recursive feature elimination with cross-validation (RFECV) [33] to identify the most influential features. RFECV works by iteratively removing features with the lowest importance scores and evaluating the model's

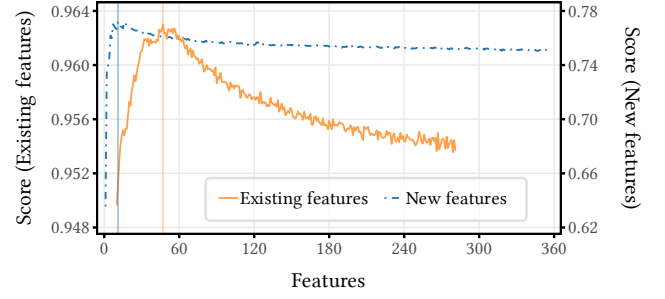


Figure 1: RFECV scores against feature count.

performance based on the remaining features. Figure 1 shows the results of the RFECV process, with the x-axis denoting the number of features and the y-axis indicating the cross-validation accuracy. The scores for the new features are lower than those for the existing features. For the existing features, accuracy increases as we remove features until we retain 47, after which it decreases. For the new features, optimal results are achieved with 11 features. Therefore, we considered a set of 58 features, combining the 47 and 11 features from the two groups.

To further streamline the number of features, we aimed to select only those that exhibited minimal correlation with each other. Thus, we analyzed the correlation for all possible pairs within these 58 features. We employed Pearson's coefficient for pairs with continuous values and used Spearman's correlation otherwise. If the correlation  $p$ -value between any two features was below 0.05, we discarded the less significant feature based on RFECV scores. This process resulted in a final set of 27 features. The comprehensive list of features in AdFlush is detailed in Appendix B.

We used a Random Forest to assess the importance of each feature in the final 27 features. We distinguished ads, trackers, and regular resources using three features: the number of storage elements retrieved by the resource, the URL length, and the number of cookie values retrieved. These features are crucial for identifying ads and trackers because they often store user identifiers in storage elements, have long request URLs, and use more cookies for tasks such as cookie synchronization. We observed a variation in the distribution of these features in our dataset, with ads and trackers typically having higher average values for all three features (see Figure 2). For example, ad and tracker resources sometimes accessed storage elements over 300 times, while regular resources rarely did so. While some features might have a minor individual impact, their collective use produces a robust indicator for differentiating ads from trackers. We found 61,285 ads and trackers that receive cookie values, access storage elements, and have a URL length of over 200 characters, in contrast to only 6,793 instances for regular resources. This suggests that these features can be used to distinguish between ad/tracker and regular resources.

Table 1 shows the relative importance of the top five newly introduced JavaScript features in terms of information gain. The most important feature is the 3-gram pattern (*Statement*, *CatchClause*, *Statement*), which is typically represented by try-catch blocks. The other top features, (*Expression*, *Identifier*, *Identifier*) and (*Expression*, *Expression*, *Statement*), are less distinctive. However, they still contribute to the model's performance, with information gains of

\*The feature counts mentioned here do not align with the number of ● symbols in Appendix A. This discrepancy occurs because we group together features computed using similar methods in the table.

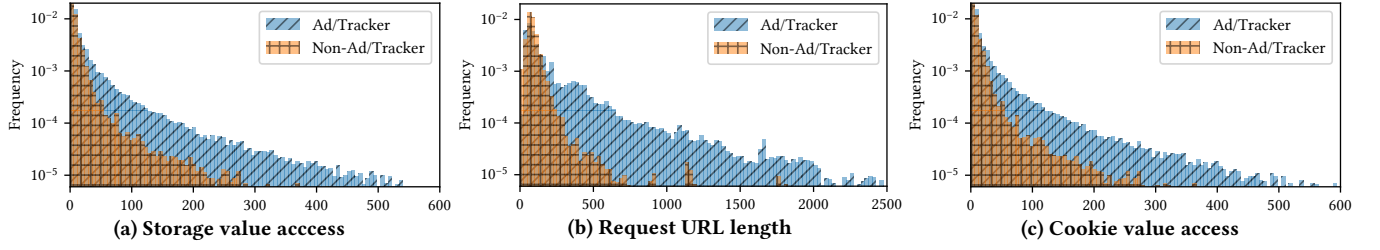


Figure 2: Distribution of three features for ad/tracker versus non-ad/tracker resources (normalized, y-axis in log scale).

Table 1: Top-5 important JavaScript features and information gain within optimized 27 features (averaged over 10 folds).

| Feature                              | Information Gain (%) |
|--------------------------------------|----------------------|
| (Statement, CatchClause, Statement)  | $5.42 \pm 0.54$      |
| Average of Identifier length         | $2.24 \pm 0.32$      |
| Average of character per line        | $1.32 \pm 0.22$      |
| (Expression, Identifier, Identifier) | $1.27 \pm 0.19$      |
| (Expression, Expression, Statement)  | $1.26 \pm 0.20$      |

1.27% ( $\pm 0.19$ ) and 1.26% ( $\pm 0.20$ ), ranking 18th and 19th in importance, respectively. Additionally, obfuscation-related features, such as *Identifier* lengths in ASTs and character counts in code lines, would also enhance performance.

## 4 IMPLEMENTATION OF ADFLUSH

We aim to identify the most effective features for improving the performance of AdFlush while minimizing its complexity. To achieve this goal, we carefully selected 27 features with the highest importance scores. We then applied a dimensionality reduction technique to this compact feature set, further simplifying the model and improving its predictive power by preventing overfitting.

To find the most efficient models for these reduced feature spaces, we used H2O AutoML [8]. This tool provides a comprehensive search and selection platform, allowing us to explore different models to identify those that provide the highest performance. Once we had identified the best-performing models, we optimized them to be more efficient and resource-friendly.

We integrate our model into a fully functional web browser extension for Chrome. This extension captures features in real-time and promptly blocks ads and trackers detected by the model, ensuring a secure browsing experience (see Figure 3).

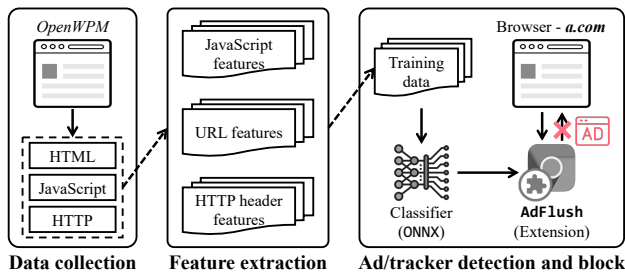


Figure 3: Overview of AdFlush.

### 4.1 Dataset

We aggregated eight popular filter lists (see Appendix C) to label our dataset accurately and distinguish between the presence or absence of ads and trackers. Our labeling method mirrors that used in existing studies [6, 22], ensuring consistency and comparability. After labeling, we divided the data into training and test sets in an 8:2 ratio. This substantial labeled dataset was then used to train and test the detection model and to compare it with state-of-the-art solutions.

### 4.2 Model Selection and Training

We used H2O AutoML [8], an automated machine learning platform, to find the best model with our feature set. AutoML facilitates the search for high-performing models and the generation of optimized feed-forward artificial neural network models. We analyzed the training data for each feature selection, identified the top 10 models based on F1 scores, and selected the best-performing model using 5-fold cross-validation. Among the top 10 models, the gradient boosting machine (GBM) [34] and Random Forest achieved the highest F1 score of 0.97. However, GBM (5.9 seconds) was significantly faster in training times than Random Forest (13.2 seconds), so we selected GBM as the best-performing model. We observed a significant drop in F1 scores below 0.95 for the other models.

For AdFlush, we selected 27 features based on their importance scores and correlation (see Section 3.4). However, our analysis found significant correlations among these features. We could use dimensionality reduction techniques to streamline the feature set and model while preserving pivotal information. We considered two dimensionality reduction techniques: Principal Component Analysis (PCA) [35] and Uniform Manifold Approximation and Projection (UMAP) [36]. While PCA is a robust technique, it can overlook subtle nuances, particularly in datasets with non-linear characteristics. To address this, we also considered UMAP. We condensed the 27 features with PCA to 8, and with UMAP, we reduced them to 2. These results are consistent with our correlation analysis, suggesting that the 27 features may contain redundant data, with some being highly interrelated.

We trained the GBM with each of these three feature sets to determine the optimal feature set and evaluated the performance of AdFlush on the test set (see Table 2).

Although reducing AdFlush's feature set to two features using UMAP achieved the highest F1 score of 0.99, we chose to retain the original 27 features because using UMAP would have complicated result interpretation and increased computational and memory overhead due to the additional processing steps. Since the performance improvement was marginal (0.99 vs. 0.98), we prioritized interpretability and efficiency and used the original features.



**Table 2: Performance of AdFlush with each feature set: Original (27), PCA (8), and UMAP (2).**

| Feature Set           | Acc.        | Prec.       | Recall      | F <sub>1</sub> |
|-----------------------|-------------|-------------|-------------|----------------|
| AdFlush Original (27) | <b>0.98</b> | <b>0.99</b> | 0.97        | 0.98           |
| AdFlush PCA (8)       | 0.96        | 0.96        | 0.93        | 0.95           |
| AdFlush UMAP (2)      | <b>0.98</b> | <b>0.99</b> | <b>0.98</b> | <b>0.99</b>    |

### 4.3 Browser Extension Implementation

Google’s recent adoption of Manifest V3 [37] restricts Chrome extensions from directly manipulating user HTTP requests. Therefore, extensions must define rules or conditions for blocking requests rather than controlling them. To comply with this new standard, we implemented AdFlush, a Chrome extension that uses a GBM model to generate rules on the fly to block ads and web trackers.

The adoption of Manifest V3 emphasizes the importance of performing model inference in real-time to generate rules. We used the libraries `onnxmltools` and `ONNX-simplifier` to convert our GBM model into the ONNX format, which enables efficient execution through a web assembly backend. Our extension for the Chrome web browser uses this model to detect ads and web trackers from web page requests in real-time.

Appendix D illustrates the detailed workflow: ① AdFlush captures network requests from the user’s active web page. ② A static rule is applied for each request. ③, and the request is redirected to the Requestly delay API. Before this, AdFlush removes user cookies and appends a dummy User-Agent to safeguard user privacy. In this step, AdFlush extracts features like content policy type, domain party, and  $n$ -gram frequencies. It references a pre-trained vector dictionary in JSON format for URL word embedding features. ④ The service worker forwards these values to the ONNX model loaded in an off-screen tab. ⑤ The model predicts and notifies the service worker based on these values. ⑥ If identified as an ad or tracker URL, a dynamic rule is established in declarativeNetRequest; otherwise, benign requests are allowed. ⑦ After the Requestly timeout, the original request is reinitiated. ⑧ Requests violating user privacy (i.e., ad or tracker URLs) are blocked.

AdFlush relies on the deprecated `onBeforeSendHeaders` function from the `webRequest` package. The latest `declarativeNetRequest` package limits the ability to read and modify raw network traffic, posing challenges for specific machine learning models that require this capability. The ability to read traffic is still permitted through `webRequest`, allowing us to deploy our AdFlush extension on the latest 122 version of Chrome. We note that during the feature extraction and model prediction process, the original request must be delayed and not forwarded to ad and tracker servers to ensure user privacy. This delay, previously available in `webRequest` with manipulation permissions, is now deprecated, necessitating a workaround through the Requestly API.

## 5 EVALUATION

We conducted a comprehensive evaluation to demonstrate the efficacy and efficiency of AdFlush. This included comparing AdFlush to state-of-the-art models, assessing its robustness to adversarial inputs, and measuring the runtime overhead of the browser extension.

**Table 3: Comparison of model performance.**

| Model        | # features    | Acc.        | Prec.       | Recall      | F <sub>1</sub> |
|--------------|---------------|-------------|-------------|-------------|----------------|
| AdGraph [5]  | 28            | 0.94        | 0.92        | 0.94        | 0.93           |
| WebGraph [6] | 59            | 0.92        | 0.91        | 0.90        | 0.90           |
| WTAGraph [7] | 3,266         | 0.90        | 0.85        | 0.83        | 0.84           |
| RandomForest | 27            | 0.98        | 0.98        | 0.96        | 0.97           |
| SkopeRules   | 27            | 0.83        | 0.87        | 0.68        | 0.77           |
| XGBoost      | 27            | 0.96        | 0.97        | 0.94        | 0.95           |
| CatBoost     | 27            | 0.95        | 0.95        | 0.92        | 0.94           |
| LightGBM     | 27            | 0.94        | 0.94        | 0.91        | 0.92           |
| AdFlush      | 28 (AdGraph)  | 0.93        | 0.90        | 0.92        | 0.91           |
| AdFlush      | 59 (WebGraph) | 0.89        | 0.85        | 0.87        | 0.86           |
| AdFlush      | 27            | <b>0.99</b> | <b>0.99</b> | <b>0.98</b> | <b>0.98</b>    |

### 5.1 Comparison with State-of-the-Art Models

To evaluate AdFlush’s detection capability, we compared it with state-of-the-art ad and tracker detection systems (AdGraph [5], WebGraph [6], and WTAGraph [7]). Additionally, we compared it with various machine learning models, including Random Forest, SkopeRules, XGBoost, CatBoost, and LightGBM, using the 27 features identified in AdFlush. Since the source code for feature extraction was not publicly available for WTAGraph, we re-implemented the feature extraction module based on the details provided in the paper. We excluded PageGraph [23] in the comparison because it relies on images within web pages to detect ads. To verify the importance of these features, we also built GBM models in AdFlush using the features from AdGraph and WebGraph, respectively, and analyzed their performance. All models were evaluated on a dataset comprising 20% of our collected data.

Table 3 shows that AdFlush outperforms other solutions in performance. Notably, AdFlush achieved an F1 score of 0.98, significantly higher than the second-best, AdGraph, which scored 0.93. In detail, AdFlush reached a True Positive Rate (TPR) of 0.976 at a False Positive Rate (FPR) of  $2.76e-5$ , confirming its reliable performance. Additionally, we found that the 27 features used in AdFlush are highly effective, as evidenced by the Random Forest, XGBoost, CatBoost, and LightGBM models also achieving F1 scores exceeding 0.9. However, the SkopeRules model exhibited poor performance with an F1 score of 0.77. When we replaced the 27 features identified in AdFlush with the 28 features from AdGraph and the 59 features from WebGraph, the performance yielded F1 scores of 0.91 and 0.86, respectively, demonstrating the significance of our chosen 27 features in achieving high detection accuracy.

We evaluated AdFlush’s performance across 14 HTTP request types (see Appendix E). Remarkably, AdFlush achieved an F1 score over 0.93 for all 14 types, demonstrating its effectiveness across all HTTP request types. AdFlush also performed efficiently for types with fewer than 1,000 requests.

### 5.2 Longitudinal Study

Websites constantly evolve, with features frequently updated due to redesigns, new content additions, or algorithm changes targeting ad serving or user tracking. As a result, ML-based solutions typically require periodic retraining on fresh datasets to maintain their performance. However, we show that AdFlush maintains its effectiveness over extended durations without retraining.

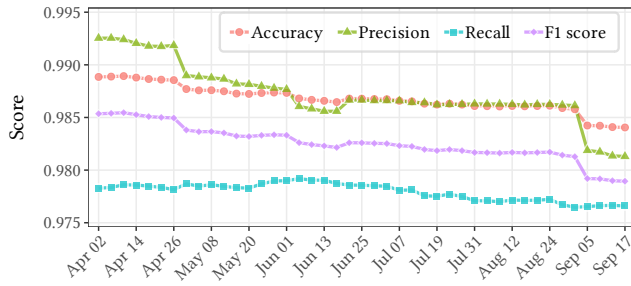


Figure 4: Longitudinal study from April to September 2023.

We conducted a longitudinal study to evaluate AdFlush’s persistent robustness and reliability, especially compared to frequently updated filter lists. We collected eight popular filter lists, updated every four days for six months, following EasyList’s update schedule from April 2, 2023, to September 17, 2023. After training AdFlush on the initial training dataset, we assessed its performance against the evolving filter lists, simulating a real-world post-launch scenario.

The performance trajectory of our detection model on the test dataset is illustrated in Figure 4. Impressively, without retraining, AdFlush exhibited robust detection capabilities throughout, maintaining an F1 score above 0.9789. Notably, the F1 score increased until April 10 and then decreased by a negligible 0.0064 over the entire period, demonstrating AdFlush’s enduring consistency over extended intervals.

AdFlush’s precision gradually declined over time, but its recall rate remained high. This suggests that AdFlush may have increased false positives, but it remained effective at detecting most ads and trackers, even when trained on an older dataset. Appendix F provides a detailed analysis of false negative samples of filter lists.

Commercial filter lists experience a significant delay in adding newly identified ads and trackers (mean = 80 days, standard deviation = 29.89 days) compared to AdFlush. Additionally, AdFlush can detect ads and trackers that are overlooked by even the most recent filter lists as of October 11, 2023. To confirm such findings, three researchers independently investigated the URLs by examining and executing JavaScript, monitoring cookies and network requests, and conducting cross-verification for validation. Through this process, we identified 642 ads and trackers from 108 unique domains and reported our findings to EasyList, EasyPrivacy, uBlock Origin, and uBlock Privacy. These actions underscore AdFlush’s utility as an effective tool for the timely update of filter lists.

### 5.3 Robustness against Adversarial Samples

ML models require rigorous validation to ensure their resilience to unseen data and feature-targeted attacks. We evaluated AdFlush’s robustness against AdGraph [5] and WebGraph [6], excluding WTA-Graph [7] due to its larger feature set and lower effectiveness than WebGraph. We used attack success rate (ASR) and AUROC as experiment metrics. Our experiments cover three mutation scenarios:

**Case 1: URL Manipulation.** Attackers with access to filter lists and a broad understanding of URL patterns used in ad and tracker detection can manipulate URLs to bypass ad blockers. They can change the domain or subdomain name, hide ad and tracker keywords within the query string, or combine these techniques. For

Table 4: Robustness against adversarial samples.

| Method                   | Model          | Acc.        | Prec.       | Recall      | F <sub>1</sub> | ASR         | AUROC       |
|--------------------------|----------------|-------------|-------------|-------------|----------------|-------------|-------------|
| URL manipulation         | AdGraph [5]    | 0.89        | 0.85        | 0.86        | 0.85           | 0.11        | 0.95        |
|                          | <b>AdFlush</b> | <b>0.98</b> | <b>0.97</b> | <b>0.96</b> | <b>0.97</b>    | <b>0.02</b> | <b>0.98</b> |
| JavaScript-obfuscator    | WebGraph [6]   | 0.86        | 0.87        | 0.76        | 0.81           | 0.14        | 0.92        |
|                          | <b>AdFlush</b> | <b>0.95</b> | <b>0.96</b> | <b>0.91</b> | <b>0.94</b>    | <b>0.05</b> | <b>0.94</b> |
| gnirts                   | WebGraph [6]   | 0.87        | 0.88        | 0.75        | 0.81           | 0.13        | <b>0.92</b> |
|                          | <b>AdFlush</b> | <b>0.92</b> | <b>0.99</b> | <b>0.81</b> | <b>0.89</b>    | <b>0.08</b> | 0.90        |
| Wobfuscator              | WebGraph [6]   | 0.86        | 0.87        | 0.76        | 0.81           | 0.14        | 0.92        |
|                          | <b>AdFlush</b> | <b>0.95</b> | <b>0.96</b> | <b>0.92</b> | <b>0.94</b>    | <b>0.05</b> | <b>0.95</b> |
| CTGAN-based manipulation | AdGraph [5]    | 0.74        | 0.78        | 0.85        | 0.81           | 0.26        | 0.75        |
|                          | WebGraph [6]   | 0.89        | 0.88        | 0.87        | 0.87           | 0.11        | 0.96        |
|                          | <b>AdFlush</b> | <b>0.98</b> | <b>0.98</b> | <b>0.97</b> | <b>0.98</b>    | <b>0.02</b> | <b>0.98</b> |

example, an attacker might manipulate the URL of an ad from <https://secure.adnxs.com/seg?ad=229&t=1> to <https://UJu0xD.OBGFcuxmU.com/seg?tk=229&62Jo=M1lrD0yUI>. In our evaluation, we applied all possible combinations of URL manipulation to our test dataset.

**Case 2: JavaScript Obfuscation.** To mimic the tactics of ad and tracker writers attempting to evade detection, we obfuscated JavaScript code using three widely recognized tools: JavaScript-Obfuscator [38], gnirts [39], and Wobfuscator [40]. gnirts is particularly effective at mangling string literals beyond simple hexadecimal string escapes, while JavaScript-Obfuscator offers various code transformations, such as variable renaming and control flow flattening. Wobfuscator replaces parts of JavaScript code with WebAssembly modules that retain the same function. Using these tools, we obfuscated 13,695, 15,386, and 13,311 samples from our test dataset, respectively. We then evaluated the resilience of our model against JavaScript obfuscation using these subsets.

**Case 3: CTGAN-Based Manipulation.** We generated adversarial samples using the conditional tabular GAN (CTGAN) [41]. CTGAN is designed to emulate the diversity of real-world data and has shown increased resilience and adaptability across various datasets. Using CTGAN, we generated adversarial versions of our test dataset, tailored to the features of our target models. We first divided our test dataset into an 8:2 ratio for training and testing. Once trained, CTGAN created adversarial samples indistinguishable from real-world data but also designed to evade detection by the tested models.

Table 4 presents the robustness of AdFlush against adversarial samples compared to AdGraph and WebGraph. We excluded WebGraph when manipulating URL features, as it does not use them, and excluded AdGraph during JavaScript manipulations due to its reliance on a single constant JavaScript feature. Overall, AdFlush consistently demonstrated the lowest ASR and highest F1 score across all adversarial sample scenarios compared to the state-of-the-art models. Specifically, AdGraph’s F1 score dropped significantly to 0.81 against CTGAN-based manipulations, while AdFlush maintained a high F1 score of 0.98. WebGraph’s F1 score decreased to 0.81 when JavaScript was obfuscated, while AdFlush consistently held an F1 score above 0.89.

### 5.4 Runtime Overhead Evaluation

To evaluate the runtime overhead of AdFlush, we measured its time complexity in a Python environment within Ubuntu 18.04 Windows Subsystem for Linux, utilizing an Intel Core i7-9700 processor with 16 GB of memory. We calculated the average time from model

loading to test dataset inference across 10 trials. The inference time for AdFlush, at 2.3 seconds, was significantly faster than that of state-of-the-art solutions (17.4 seconds for AdGraph [5], 19.7 seconds for WebGraph [6], and 144.9 seconds for WTAGraph [7]), achieving an 86% reduction compared to AdGraph. This improved inference speed highlights AdFlush's efficiency in detection time.

AdFlush is also superior in terms of CPU and memory consumption. For CPU usage, AdGraph and WebGraph consumed 1.8% and 1.6%, respectively, while AdFlush used only 0.8%, requiring 56% less CPU than AdGraph. WTAGraph also reported 0.8% CPU usage, but its memory demand was significantly higher than AdFlush's. WTAGraph required a substantial 1.3 GB of memory during inference, while AdGraph and WebGraph needed over 200 MB and 150 MB, respectively. In comparison, AdFlush's memory footprint was a modest 40 MB, a reduction of up to 80% from AdGraph.

## 5.5 Comparison with uBlock Origin

We evaluated AdFlush's ability to detect and prevent ads and trackers as a comprehensive application in the real world. For comparison, we chose uBlock Origin [16] as a baseline, known for its capability to inject manually sanitized scriptlets to replace predefined ad and tracker-related JavaScript. We aligned uBlock Origin's release date with the training cut-off date for AdFlush and modified its source code for evaluation purposes.

We visited the top 1,000 Tranco websites using Selenium to replicate user engagements on a Chrome browser as of December 7, 2023. We labeled 51,139 traffic requests using the eight filter lists updated to December 13, 2023. AdFlush outperformed uBlock Origin with an F1 score of 0.86 compared to 0.84, demonstrating its superior detection accuracy in a real-world environment, even eight months after its last training.

Additionally, we measured the time required to load the entire page for the 1,000 websites. Without ad and tracker blocking tools, the median page load time was 2.728 seconds, with an Interquartile Range (IQR) of 7.808 seconds. With uBlock Origin, the page load time was decreased to a median of 2.161 seconds (IQR = 6.748 seconds), indicating that the overhead of filtering ads and trackers is more than compensated for by blocking resource-intensive ads and trackers. AdFlush showed a higher median of 6.648 seconds (IQR = 18.677 seconds). However, by preserving prior predictions as dynamic rules, AdFlush achieved a median page load time of 3.382 seconds (IQR = 7.928 seconds) on subsequent visits, comparable to scenarios without blocking tools.

For a single request, the median time taken by AdFlush for feature extraction was 0.081 seconds (IQR = 0.609 seconds), and for inference, it was 0.004 seconds (IQR = 0.032 seconds). Importantly, these tasks do not extend the processing time additively. Leveraging JavaScript's asynchronous functionalities, AdFlush operates concurrently without disrupting the user experience and proves to be much faster from the user's perspective. These findings align with our implementation strategy for AdFlush, which includes a 300-millisecond delay as described in 4.3.

## 6 DISCUSSION

**Validity of Ground Truth Dataset.** Machine learning models that detect ads and web trackers, such as AdFlush, often rely on

filter lists like EasyList [1] as their ground truth. The accuracy of these models is inherently linked to the reliability of these lists. Sole reliance on filter lists introduces two primary challenges: (1) The need for frequent updates to ensure that the lists are comprehensive and up-to-date, and (2) potential inaccuracies in the lists that could jeopardize the model's trustworthiness. Le *et al.* [26] highlighted these issues and proposed AdHighlighter as a solution; however, this tool also requires regular maintenance. As described in Section 4.1, to mitigate these challenges, we used eight popular filter lists (detailed in Appendix C) to obtain valid labels for our collected dataset. This approach helped reduce the risk of bias from relying on a single filter list.

**Real-Time Ad Blocking in Browser Extensions.** ML-based ad and web tracker detectors often emphasize model accuracy without thoroughly considering the feasibility of implementing their features in real-world web browser extensions. For ad-blocking extensions, the primary objective is to halt requests intended for ads and web trackers in real-time. However, many graph-based features used in existing ML-based solutions require the outcomes of HTTP requests and the execution results of JavaScript code, indicating that the ad and tracker code must be executed before detection occurs. To address this challenge, we exclusively considered features that can be swiftly computed in real-time within a browser extension. This allows AdFlush to quickly generate rules for blocking URLs in real-time before HTTP requests are delivered and code is executed.

**Privacy Limitation of AdFlush.** As mentioned in Section 4.3, AdFlush currently removes UserAgent and cookie values from network requests before sending them to a third-party server (i.e., Requestly) to protect user privacy. However, this method may not fully address all privacy concerns, particularly regarding sensitive URLs and IP addresses. In future work, we plan to use a local proxy server instead of relying on a third-party server. This approach will allow for the internal processing of requests, significantly improving user privacy protection.

## 7 CONCLUSION

We have developed AdFlush, a novel framework specifically tailored for detecting ads and trackers, incorporating 27 optimally selected features that surpass the capabilities of existing tools.

On a benchmark of the top 10,000 websites, AdFlush achieved an F1 score of 0.98, outperforming the leading state-of-the-art tool, AdGraph, by a margin of 0.05. Additionally, AdFlush demonstrated resilience against adversarial attacks, outperforming other methods. Remarkably, AdFlush maintained its high detection accuracy over six months without retraining, showcasing its robustness and sustained effectiveness against new and unseen data.

## ACKNOWLEDGEMENT

We thank the anonymous reviewers for their constructive comments. Hyoungshick Kim is the corresponding author. This work was supported by the Korea Internet & Security Agency (KISA) grant (No.1781000003), and the Institute of Information & communications Technology Planning & Evaluation (IITP) grants (No.2022-0-00688, 2022-0-01199, and RS-2023-00229400) funded by the Korean government.



## REFERENCES

- [1] Easylist. URL: <https://easylist.to/>.
- [2] Easyprivacy. URL: <https://easylist.to/easylist/easyprivacy.txt>.
- [3] Mshabab Alrizah, Sencun Zhu, Xinyu Xing, and Gang Wang. Errors, misunderstandings, and attacks: Analyzing the crowdsourcing process of ad-blocking systems. In *Proceedings of the 2019 Internet Measurement Conference (IMC)*, pages 230–244, 2019.
- [4] Umar Iqbal, Steven Englehardt, and Zubair Shafiq. Fingerprinting the fingerprinters: Learning to detect browser fingerprinting behaviors. In *Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP)*, pages 1143–1161, 2021.
- [5] Umar Iqbal, Peter Snyder, Shitong Zhu, Benjamin Livshits, Zhiyun Qian, and Zubair Shafiq. AdGraph: A graph-based approach to ad and tracker blocking. In *Proceedings of the 2020 IEEE Symposium on Security and Privacy (SP)*, pages 763–776, 2020.
- [6] Sandra Siby, Umar Iqbal, Steven Englehardt, Zubair Shafiq, and Carmela Troncoso. WebGraph: Capturing advertising and tracking information flows for robust blocking. In *Proceedings of the 2022 USENIX Security Symposium (Security)*, pages 2875–2892, 2022.
- [7] Zhijun Yang, Weiping Pei, Monchu Chen, and Chuan Yue. WTAGraph: Web tracking and advertising detection using graph neural networks. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*, pages 1540–1557, 2022.
- [8] Erin LeDell and Sebastien Poirier. H2O automl: Scalable automatic machine learning. In *Proceedings of the 2020 Workshop on Automatic Machine Learning (ICML)*, volume 2020, 2020.
- [9] Jonathan R Mayer and John C Mitchell. Third-party web tracking: Policy and technology. In *Proceedings of the 2012 IEEE symposium on security and privacy (SP)*, pages 413–427, 2012.
- [10] Brian X. Chen. The battle for digital privacy is reshaping the internet, Sep 2021. URL: <https://www.nytimes.com/2021/09/16/technology/digital-privacy.html>.
- [11] Your data is shared and sold... What's being done about it?, Oct 2019. URL: <https://knowledge.wharton.upenn.edu/article/data-shared-sold-whats-done/>.
- [12] Mark Yep-Kui Chua, George OM Yee, Yuan Xiang Gu, and Chung-Horng Lung. Threats to online advertising and countermeasures: A technical survey. *Digital Threats: Research and Practice*, 1(2):1–27, 2020.
- [13] Tom Hegel. Breaking down the SEO poisoning attack: How attackers are hijacking search results, Jan 2023. URL: <https://www.sentinelone.com/blog/breaking-down-the-seo-poisoning-attack-how-attackers-are-hijacking-search-results/>.
- [14] Steven Englehardt, Dillon Reisman, Christian Eubank, Peter Zimmerman, Jonathan Mayer, Arvind Narayanan, and Edward W. Felten. Cookies that give you away: The surveillance implications of web tracking. In *Proceedings of the 2015 International Conference on World Wide Web (WWW)*, pages 289–299, 2015.
- [15] Georg Merzdovnik, Markus Huber, Damjan Buhov, Nick Nikiforakis, Sebastian Neuner, Martin Schmiedecker, and Edgar Weippl. Block me if you can: A large-scale study of tracker-blocking tools. In *Proceedings of 2017 IEEE European Symposium on Security and Privacy (Euro S&P)*, pages 319–333, 2017.
- [16] Raymond Hill. Unblock origin, 2020. URL: <https://ublockorigin.com/>.
- [17] Privacy Badger. URL: <https://privacybadger.org/>.
- [18] Disconnect. URL: <https://disconnect.me/>.
- [19] Firefox. URL: <https://www.mozilla.org/en-US/firefox/features/private-browsing/>.
- [20] Brave browser. URL: <https://brave.com/>.
- [21] Fanboy list. URL: <https://fanboy.co.nz/>.
- [22] Umar Iqbal, Zubair Shafiq, and Zhiyun Qian. The ad wars: retrospective measurement and analysis of anti-adblock filter lists. In *Proceedings of the 2017 Internet Measurement Conference (IMC)*, pages 171–183, 2017.
- [23] Alexander Sjösten, Peter Snyder, Antonio Pastor, Panagiotis Papadopoulos, and Benjamin Livshits. Filter list generation for underserved regions. In *Proceedings of the 2020 Web Conference (WWW)*, pages 1682–1692, 2020.
- [24] Steven Englehardt and Arvind Narayanan. Online tracking: A 1-million-site measurement and analysis. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.
- [25] Sruti Bhagavatula, Christopher Dunn, Chris Kanich, Minaxi Gupta, and Brian Ziebart. Leveraging machine learning to improve unwanted resource filtering. In *Proceedings of the 2014 Workshop on Artificial Intelligence and Security Workshop (AISec)*, pages 95–102, 2014.
- [26] Hieu Le, Salma Elmalaki, Athina Markopoulou, and Zubair Shafiq. AutoFR: Automated filter rule generation for adblocking. In *Proceedings of the 2023 USENIX Security Symposium (Security)*, pages 7535–7552, 2023.
- [27] Grant Storey, Dillon Reisman, Jonathan Mayer, and Arvind Naayana. The future of ad blocking: An analytical framework and new techniques. *arXiv preprint arXiv:1705.08568*, 2017.
- [28] Zainul Abi Din, Panagiotis Tigas, Samuel T. King, and Benjamin Livshits. Percival: Making in-browser perceptual ad blocking practical with deep learning. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC)*, pages 387–400, 2020.
- [29] Florian Tramèr, Pascal Dupré, Gili Rusak, Giancarlo Pellegrino, and Dan Boneh. Adversarial: Perceptual ad blocking meets adversarial machine learning. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2005–2021, 2019.
- [30] Victor Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczynski, and Wouter Joosen. Tranco: A research-oriented top sites ranking hardened against manipulation. In *Proceedings of the 2019 Network and Distributed System Security Symposium (NDSS)*, 2019.
- [31] Kunlun Ren, Weizhong Qiang, Yueming Wu, Yi Zhou, Deqing Zou, and Hai Jin. An empirical study on the effects of obfuscation on static machine learning-based malicious javascript detectors. In *Proceedings of the 2023 ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1420–1432, 2023.
- [32] Diana Kornbrot. Point biserial correlation. *Wiley StatsRef: Statistics Reference Online*, 2014.
- [33] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine learning*, 46:389–422, 2002.
- [34] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P Adams, and Nando De Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2015.
- [35] Caterina Labrin and Francisco Urdinez. Principal component analysis. In *R for Political Data Science*, pages 375–393. Chapman and Hall/CRC, 2020.
- [36] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- [37] Chrome.declarativenetrequest. URL: <https://developer.chrome.com/docs/extensions/reference/declarativeNetRequest/>.
- [38] Javascript-obfuscator: A powerful obfuscator for javascript and node.js. URL: <https://github.com/javascript-obfuscator/javascript-obfuscator>.
- [39] Gnirts: Obfuscate string literals in javascript code. URL: <https://github.com/anseki/gnirts>.
- [40] Alan Romano, Daniel Lehmann, Michael Pradel, and Weihang Wang. Wobfuscator: Obfuscating javascript malware via opportunistic translation to webassembly. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP)*, pages 1574–1589, 2022.
- [41] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. Modeling tabular data using conditional gan. *Advances in neural information processing systems*, 32, 2019.
- [42] Peterlowe's list. URL: <https://pgl.yoyo.org/adserver/serverlist.php?hostformat=adblockplus>.
- [43] Warning removal list. URL: <https://easylist-downloads.adblockplus.org/antiadblockfilters.txt>.

## A CATEGORIZATION OF CANDIDATE FEATURES

Table 5 shows the features that we investigate in this paper, including all aggregated features from AdGraph [5], WTAGraph [7], and WebGraph [6], as well as new features against JavaScript obfuscation strategies. Duplicate features are grouped by implementation, and we group the entire feature set into JavaScript, URL, HTTP header, and graph categories based on their resource. Specific features considered for AdFlush are marked with “‡” in the table.

subsection 3.3 explores the feasibility of implementing each feature within a privacy-preserving browser environment. Following our rules to ensure users’ privacy, we note whether the feature is secure to implement in a commercial web browser. Only one feature in the Graph category is available by observing the Initiator in the request header without referencing other requests or response headers. For the same reason, the *max\_depth\_of\_redirect* is only observable by chaining requests and responses with a communicated server. Therefore, extracting this feature is vulnerable to ad and tracker collecting user privacy.

## B FEATURE SET OF ADFLUSH

AdFlush uses a carefully selected collection of the most compelling features to detect ads and web trackers. These features are detailed in Table 6 and are used in both Python and web browser environments. For example, we compute the number of requests sent, storage values set or fetched, and accesses of cookies based on the

**Table 5: Summary of the categorized features. Feature sets include AdGraph (A), WTAGraph (T), and WebGraph (W), with AdFlush ( $\ddagger$ ) denoting new features. A ‘●’ indicates a tool’s use of the feature, while ‘✓’ signifies the feasibility (F) of integrating the feature into a browser extension.**

| Theme       | Feature                                                 | A | T | W | F |
|-------------|---------------------------------------------------------|---|---|---|---|
| JavaScript  | # of requests sent                                      |   | ● | ● | ✓ |
|             | # of storage (set/get)                                  |   | ● | ● | ✓ |
|             | # of cookies (set/get)                                  |   | ● | ● | ✓ |
|             | Use of <i>eval/function</i> instructions                | ● | ● | ● | ✓ |
|             | Use of ad/tracker related JavaScript API                |   | ● |   | ✓ |
|             | <i>n</i> -gram frequency of JavaScript AST $\ddagger$   |   |   |   | ✓ |
|             | JavaScript AST depth/breadth $\ddagger$                 |   |   |   | ✓ |
|             | Average of <i>Identifier</i> length $\ddagger$          |   |   |   | ✓ |
|             | Average of characters per line $\ddagger$               |   |   |   | ✓ |
|             | Bracket to dot notations ratio in JavaScript $\ddagger$ |   |   |   | ✓ |
| URL         | URL length                                              | ● | ● |   | ✓ |
|             | Sub-domain check                                        | ● |   |   | ✓ |
|             | Valid query string parameters                           | ● |   |   | ✓ |
|             | Domain party (first/third)                              | ● |   |   | ✓ |
|             | Specific keywords in request URL                        | ● |   |   | ✓ |
|             | Semi-colons in request URL                              | ● |   |   | ✓ |
|             | Base domain in request URL                              | ● |   |   | ✓ |
|             | Screen size in request URL                              | ● |   |   | ✓ |
|             | Character embeddings                                    |   | ● |   | ✓ |
| HTTP header | Content policy type (categorical)                       | ● | ● |   | ✓ |
|             | Content policy type (boolean)                           |   | ● |   | ✓ |
|             | Request Content type (boolean)                          |   | ● |   | ✓ |
|             | Request Method type (boolean)                           |   | ● |   | ✓ |
|             | Order, timing, and # of attributes                      |   | ● |   | ✓ |
| Graph       | # of nodes/edges and node/edge ratio                    | ● |   | ● |   |
|             | Degree (in, out, in+out)                                | ● |   | ● |   |
|             | Average degree connectivity                             | ● |   | ● |   |
|             | Presence of ancestor script                             | ● |   | ● |   |
|             | Presence of parent script                               | ● |   | ● | ✓ |
|             | Ascendant script length                                 | ● |   | ● |   |
|             | Ad keywords in ascendant script                         | ● |   | ● |   |
|             | Use of <i>eval/function</i> in ascendant                | ● |   | ● |   |
|             | Use of <i>eval/function</i> in descendant               | ● |   | ● |   |
|             | # of ancestors/descendants                              |   |   | ● |   |
|             | Closeness centrality/eccentricity                       |   |   | ● |   |
|             | # of predecessors/successors script                     |   |   | ● |   |
|             | # of requests received                                  |   |   | ● |   |
|             | # of redirects sent/received                            |   |   | ● |   |
|             | Max depth of redirect                                   |   |   | ● |   |
|             | # of indirect degree (in/out)                           |   |   | ● |   |
|             | # of indirect ancestors/descendants                     |   |   | ● |   |
|             | Indirect centrality/connectivity                        |   |   | ● |   |
|             | Indirect in/out weights (min/max/mean)                  |   |   | ● |   |
|             | # of set then get/modify storage (src/dst)              |   |   | ● |   |
|             | # of set/get URL (src/dst)                              |   |   | ● |   |
|             | # of all indirect degree (in/out)                       |   |   | ● |   |
|             | # of all indirect ancestors/descendants                 |   |   | ● |   |
|             | Indirect all centrality/connectivity                    |   |   | ● |   |

occurrences of related JavaScript API calls, similar to how WebGraph does it. We also calculate the average length of *Identifiers* by averaging the length of names parsed as *Identifier* tokens within JavaScript source code and HTML `<script>` tags. Finally, we use a Boolean feature to indicate whether a unique character follows an ad or tracker keyword within the request URL.

Overall, AdFlush uses various features to detect ads and web trackers. These features are carefully selected to be effective and efficient, and they are implemented to preserve user privacy.

**Table 6: Features used in AdFlush. The types stand for Numerical (N), Boolean (B), and Categorical (C).**

| Category     | Feature                                     | Type | # of Values |
|--------------|---------------------------------------------|------|-------------|
| JavaScript   | # of requests sent                          | N    | 1           |
|              | # of storage set                            | N    | 1           |
|              | # of storage get                            | N    | 1           |
|              | # of cookies get                            | N    | 1           |
|              | <i>n</i> -gram frequency of JavaScript AST  | N    | 6           |
|              | Average of <i>Identifier</i> length         | N    | 1           |
|              | Average of characters per line              | N    | 1           |
| URL          | Bracket to dot notation ratio in JavaScript | N    | 1           |
|              | Third-party check                           | B    | 1           |
|              | Request URL length                          | N    | 1           |
|              | Ad keyword with special character in URL    | B    | 1           |
|              | FQDN character embeddings                   | N    | 7           |
|              | Request URL character embeddings            | N    | 3           |
| HTTP header  | Content policy type                         | C    | 1           |
| <b>Total</b> | -                                           | -    | <b>27</b>   |

## C DATASET LABELING WITH FILTER LISTS

We aggregated continually updated filter lists to label our dataset, which served as our ground truth dataset. We collected the filter lists on April 4, 2023, using the same methodology as Iqbal *et al.* [5] and Siby *et al.* [6]. However, Anti-Adblock Killer, Blockzilla, and Squid Blacklist were no longer actively updated or supported, so we incorporated two additional filter lists, uBlock Origin, and uBlock Privacy, to bolster our ground truth dataset. Table 7 shows the lists used in AdFlush, along with the number of rules in each list as of the collection date and the source of each filter list. By integrating filter lists beyond those specified in Table 7, AdFlush’s performance can potentially improve due to more reliable labels.

**Table 7: Filter lists for labeling.**

| Filter List (# Rules)           | URL                                                                                                                                                                               |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EasyList [1] (53,997)           | <a href="https://easylist.to/easylist/easylist.txt">https://easylist.to/easylist/easylist.txt</a>                                                                                 |
| EasyPrivacy [2] (30,646)        | <a href="https://easylist.to/easylist/easyprivacy.txt">https://easylist.to/easylist/easyprivacy.txt</a>                                                                           |
| Fanboy annoyance [21] (4,644)   | <a href="https://easylist.to/easylist/fanboy-annoyance.txt">https://easylist.to/easylist/fanboy-annoyance.txt</a>                                                                 |
| Fanboy social [21] (18,883)     | <a href="https://easylist.to/easylist/fanboy-social.txt">https://easylist.to/easylist/fanboy-social.txt</a>                                                                       |
| Peterlowe [42] (3,667)          | <a href="https://pgl.yoyo.org/adserver/serverlist.php?hostformat=adblockplus">https://pgl.yoyo.org/adserver/serverlist.php?hostformat=adblockplus</a>                             |
| uBlock Origin [16] (11,152)     | <a href="https://raw.githubusercontent.com/uBlockOrigin/uAssets/master/filters/filters.txt">https://raw.githubusercontent.com/uBlockOrigin/uAssets/master/filters/filters.txt</a> |
| uBlock Privacy [16] (219)       | <a href="https://raw.githubusercontent.com/uBlockOrigin/uAssets/master/filters/privacy.txt">https://raw.githubusercontent.com/uBlockOrigin/uAssets/master/filters/privacy.txt</a> |
| Warning removal list [43] (697) | <a href="https://easylist-downloads.adblockplus.org/antiadblockfilters.txt">https://easylist-downloads.adblockplus.org/antiadblockfilters.txt</a>                                 |

## D BROWSER EXTENSION IMPLEMENTATION

Figure 5 illustrates the workflow of AdFlush’s browser extension. It captures network requests from the web page visited by the user and applies a redirect rule to the Requestly delay API. To protect user privacy, AdFlush removes cookies and attaches a dummy User-Agent during the redirection. Meanwhile, features are extracted and input into the ONNX model running in an off-screen tab. If a request is identified as an ad or tracker, it is blocked; otherwise, benign requests are allowed to proceed as normal.

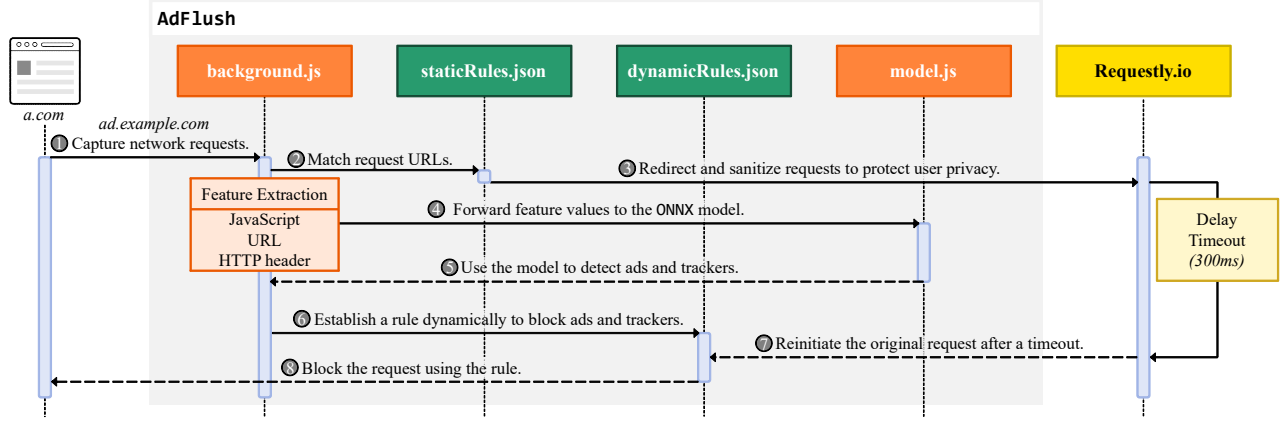


Figure 5: Sequence diagram of AdFlush in browser extension.

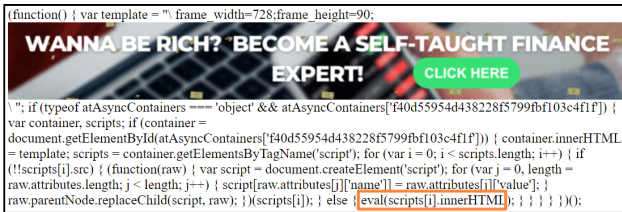


Figure 6: Script detected by AdFlush that the filter lists missed.

## E HTTP REQUEST COVERAGE OF ADFLUSH

Table 8 provides a detailed breakdown of AdFlush’s detection performance across 14 types of HTTP requests in our Top-10K dataset. AdFlush captures all 14 request types, as outlined in Google Chrome’s documentation.

As the table shows, AdFlush identifies a similar number of HTTP requests as the filter lists for each request type. Overall, AdFlush and filter lists detect 39.27% and 39.92% of all requests as advertisements and web trackers, respectively, demonstrating that AdFlush can effectively serve as a substitute for filter lists.

Notably, AdFlush demonstrates exceptional proficiency across the majority of request categories. For instance, it achieves an accuracy, precision, recall, and F1 score exceeding 0.99 for the three most common request types, image, script, and XMLHttpRequest, which collectively constitute 80.41% of all dataset requests.

However, AdFlush’s performance is somewhat diminished for imageset, main\_frame, and other request types, with false negative rates of 0.06, 0.13, and 0.09, respectively. These request types have a skewed distribution of true and false values, with only 1.75%, 3.60%, and 3.30% true values per the filter lists.

## F ADS AND TRACKERS MISSED BY FILTER LISTS

In our longitudinal study detailed in subsection 5.2, we identified significant samples of advertisements and trackers mislabeled by even commercial filter lists, but that AdFlush could detect. Under thorough analysis, we verified several patterns of ads and trackers that commercial filter lists could not detect.

```

1 t.prototype.assembleData = function (e) {
2   var t = this
3   .beaconSession
4   .getSession();
5   return {
6     appVersion: this.config.appVersion ?
7       Object(s.e)(this.config.appVersion) : "",
8     sdkId: "js",
9     sdkVersion: "4.3.4-web",
10    mainAppKey: this.config.appkey,
11    platformId: 3,
12    common: Object(s.d)(I(I({}, this.additionalParams), {
13      A2: this.commonInfo.deviceId, A8: this.commonInfo.openid,
14      A12: this.commonInfo.language, A17: this.commonInfo.pixel,
15      A23: this.commonInfo.channelId, A50: this.commonInfo.unid,
16      A76: t.sessionId, A101: this.commonInfo.userAgent,
17      A102: window.location.href, A104: document.referrer,
18      A119: this.commonInfo.query
19    })), I1),
20    events: e
21  }
22 }
23 t.prototype.onSendBeacon = function (e, t) {
24   if (this.isUnderIE8)
25     this.errorReport.reportError("601", "UnderIE8");
26   else {
27     var n = this.assembleData(this.generateData(e, t, 10));
28     navigator.sendBeacon(this.strategy.getUploadUrl(),
29       JSON.stringify(n))
30   }
31 }
  
```

Figure 7: Example tracker code detected by AdFlush.

We found that iframe is frequently used for ads and trackers. These iframe tags contained several code snippets designed to execute JavaScript segmented into elements. An example is shown in Figure 6, where the eval() function is used to process inner HTML text as source codes within alternative containers.

AdFlush also detected other requests to JavaScript resources that were later added to conventional filter lists. Figure 7 shows an example of a detected tracker source code that transmits valuable user data to a server using sendBeacon. Notably, AdFlush could accurately detect this tracker code, which is not detected by commercial filter lists.

Finally, we present examples of tracker behavior evident in URLs and HTTP headers. Request ID 114,851, illustrated in Figure 8, is directed at an image URL but returns a single pixel. This tactic transmits user data to the tracker through the query string. Similarly, Request ID 59,437, another image URL, responds with a single pixel due to the storage of user identifiers in the browser cookie. Although filter lists overlooked these instances of privacy breaches, AdFlush efficiently mitigated them.

**Table 8: Detection performance of AdFlush by HTTP request type.**

| Request Type   | # Requests     | # Blocked by<br>Filter Lists | # Detected by<br>AdFlush | Accuracy    | Precision   | Recall      | F <sub>1</sub> | FNR         | FPR         |
|----------------|----------------|------------------------------|--------------------------|-------------|-------------|-------------|----------------|-------------|-------------|
| image          | 312,231        | 120,120                      | 119,422                  | 0.99        | 0.99        | 0.99        | 0.99           | 0.01        | 0.00        |
| script         | 237,509        | 112,772                      | 111,961                  | 0.99        | 0.99        | 0.99        | 0.99           | 0.01        | 0.00        |
| xmlhttprequest | 117,826        | 59,528                       | 59,139                   | 0.99        | 0.99        | 0.99        | 0.99           | 0.01        | 0.00        |
| stylesheet     | 43,674         | 3,449                        | 3,358                    | 0.99        | 0.99        | 0.97        | 0.99           | 0.03        | 0.00        |
| font           | 33,314         | 3,642                        | 3,626                    | 0.99        | 0.99        | 0.99        | 0.99           | 0.00        | 0.00        |
| imageset       | 26,382         | 462                          | 434                      | 0.99        | 0.99        | 0.94        | 0.97           | 0.06        | 0.00        |
| sub_frame      | 25,620         | 19,014                       | 18,985                   | 0.99        | 0.99        | 0.99        | 0.99           | 0.00        | 0.01        |
| main_frame     | 18,459         | 665                          | 573                      | 0.99        | 0.99        | 0.86        | 0.93           | 0.13        | 0.00        |
| ping           | 11,736         | 11,231                       | 11,218                   | 0.99        | 0.99        | 0.99        | 0.99           | 0.00        | 0.02        |
| media          | 1,654          | 281                          | 278                      | 0.99        | 0.99        | 0.98        | 0.99           | 0.02        | 0.00        |
| other          | 696            | 23                           | 21                       | 0.99        | 0.99        | 0.91        | 0.95           | 0.09        | 0.00        |
| websocket      | 511            | 132                          | 128                      | 0.99        | 0.99        | 0.97        | 0.98           | 0.03        | 0.01        |
| csp_report     | 398            | 58                           | 54                       | 0.99        | 0.98        | 0.93        | 0.96           | 0.07        | 0.00        |
| object         | 150            | 12                           | 12                       | 0.99        | 0.99        | 0.99        | 0.99           | 0.00        | 0.00        |
| <b>Total</b>   | <b>830,160</b> | <b>331,389</b>               | <b>329,205</b>           | <b>0.99</b> | <b>0.99</b> | <b>0.99</b> | <b>0.99</b>    | <b>0.01</b> | <b>0.00</b> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Request ID: 114851                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <pre> https://stat.tiara.tistory.com/track?d=   "sdk":{     "type":"WEB","version":"1.1.25"   },"env":{     "screen":"1512X982","tz":"+9","cke":"Y"   },"common":{     "session_timeout":"1800","svcdomain":"user.tistory.com","deployment":"production",     "url":"https://www.tistory.com/", "title":"TISTORY","section":"top","page":"user","kao_     kao_app_key":"#####"   },etc:{     "client_info":{       "tuid":"####","tsid":"####","uuid":"####","suid":"####","isuid":"####","client_       timestamp":1680553904861     }   },action:{     "type":"Pageview","name":"user","kind":""   } } </pre> |
| Request ID: 59437                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <pre> Response Header:{   Set-Cookie: ELOQUA=GUID-####; domain=dow.com; expires=Mon, 11-Nov-2024   10:33:58 GMT; path=/; secure } Request Header: {   Cookie: ELOQUA=GUID-####; AKA_A2=A; cs mk=####; at check=true; s_ips=1051;   mbox=session#####PC#####; dtCookie=####; rxVisitor=####; s_tp=2246;   s_plt=4.03; s_pltp=home; _cs_c=1; _cs_id=####; _cs_s=####; s_ecid=####;   AMCVS_988D095F54BD18520A4C98A5%40AdobeOrg=1; s_cc=true;   _hjSessionUser_####-####; _hjFirstSeen=1; } </pre>                                                                                                                 |

**Figure 8: Examples of tracker behavior in URL and HTTP headers.**

To further explore filter lists' false negatives, we relabeled our dataset with the latest filter lists from October 11, 2023. By manually verifying the newly labeled AdFlush detections, we obtained 642 URLs from 108 unique domains where all authors agreed on strict cross-verification. We reported these results to EasyList, EasyPrivacy, uBlock Origin, and uBlock Privacy to help create more effective and updated filter lists.